

jupyter-labs-eda-sql-coursera_sqllite

July 23, 2026

Assignment: SQL Notebook for Peer Assignment

Estimated time needed: **60** minutes.

0.1 Introduction

Using this Python notebook you will:

1. Understand the SpaceX DataSet
2. Load the dataset into the corresponding table in a Db2 database
3. Execute SQL queries to answer assignment questions

0.2 Overview of the DataSet

SpaceX has gained worldwide attention for a series of historic milestones.

It is the only private company ever to return a spacecraft from low-earth orbit, which it first accomplished in December 2010. SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars whereas other providers cost upward of 165 million dollars each, much of the savings is because Space X can reuse the first stage.

Therefore if we can determine if the first stage will land, we can determine the cost of a launch.

This information can be used if an alternate company wants to bid against SpaceX for a rocket launch.

This dataset includes a record for each payload carried during a SpaceX mission into outer space.

0.2.1 Download the datasets

This assignment requires you to load the spacex dataset.

In many cases the dataset to be analyzed is available as a .CSV (comma separated values) file, perhaps on the internet. Click on the link below to download and save the dataset (.CSV file):

Spacex DataSet

```
[1]: !pip install sqlalchemy==1.3.9
```

```
Requirement already satisfied: sqlalchemy==1.3.9 in  
/opt/conda/lib/python3.12/site-packages (1.3.9)
```

0.2.2 Connect to the database

Let us first load the SQL extension and establish a connection with the database !pip install ipython-sql !pip install ipython-sql prettytable

```
[8]: %load_ext sql
```

The sql extension is already loaded. To reload it, use:
%reload_ext sql

```
[9]: import csv, sqlite3
import prettytable
prettytable.DEFAULT = 'DEFAULT'

con = sqlite3.connect("my_data1.db")
cur = con.cursor()
```

```
[10]: !pip install -q pandas
```

```
[11]: import pandas as pd
df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
↳cloud/IBM-DS0321EN-SkillsNetwork/labs/module_2/data/Spacex.csv")
df.to_sql("SPACEXTBL", con, if_exists='replace', index=False, method="multi")
```

```
[11]: 101
```

Note: This below code is added to remove blank rows from table

```
[14]: #DROP THE TABLE IF EXISTS

%sql sqlite:///my_data1.db DROP TABLE IF EXISTS SPACEXTABLE;
```

Done.

```
[14]: []
```

```
[15]: %sql create table SPACEXTABLE as select * from SPACEXTBL where Date is not null
```

```
* sqlite:///my_data1.db
Done.
```

```
[15]: []
```

0.3 Tasks

Now write and execute SQL queries to solve the assignment tasks.

Note: If the column names are in mixed case enclose it in double quotes For Example “Landing_Outcome”

0.3.1 Task 1

Display the names of the unique launch sites in the space mission

```
[16]: %sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
* sqlite:///my_data1.db
Done.
```

```
[16]: [('CCAFS LC-40',), ('VAFB SLC-4E',), ('KSC LC-39A',), ('CCAFS SLC-40',)]
```

0.3.2 Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
[17]: %sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

```
* sqlite:///my_data1.db
Done.
```

```
[17]: [('2010-06-04', '18:45:00', 'F9 v1.0 B0003', 'CCAFS LC-40', 'Dragon Spacecraft
Qualification Unit', 0, 'LEO', 'SpaceX', 'Success', 'Failure (parachute)'),
('2010-12-08', '15:43:00', 'F9 v1.0 B0004', 'CCAFS LC-40', 'Dragon demo flight
C1, two CubeSats, barrel of Brouere cheese', 0, 'LEO (ISS)', 'NASA (COTS) NRO',
'Success', 'Failure (parachute)'),
('2012-05-22', '7:44:00', 'F9 v1.0 B0005', 'CCAFS LC-40', 'Dragon demo flight
C2', 525, 'LEO (ISS)', 'NASA (COTS)', 'Success', 'No attempt'),
('2012-10-08', '0:35:00', 'F9 v1.0 B0006', 'CCAFS LC-40', 'SpaceX CRS-1', 500,
'LEO (ISS)', 'NASA (CRS)', 'Success', 'No attempt'),
('2013-03-01', '15:10:00', 'F9 v1.0 B0007', 'CCAFS LC-40', 'SpaceX CRS-2',
677, 'LEO (ISS)', 'NASA (CRS)', 'Success', 'No attempt')]
```

0.3.3 Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[18]: %sql SELECT SUM("PAYLOAD_MASS__KG_") FROM SPACEXTABLE WHERE "Customer" = 'NASA_
↳(CRS)';
```

```
* sqlite:///my_data1.db
Done.
```

```
[18]: [(45596,)]
```

0.3.4 Task 4

Display average payload mass carried by booster version F9 v1.1

```
[19]: %sql SELECT AVG("PAYLOAD_MASS__KG_") FROM SPACEXTABLE WHERE "Booster_Version" =_
↳'F9 v1.1';
```

```
* sqlite:///my_data1.db
Done.
```

```
[19]: [(2928.4,)]
```

0.3.5 Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
[20]: %sql SELECT MIN("Date") FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success_
      ↳(ground pad)';
```

```
* sqlite:///my_data1.db
Done.
```

```
[20]: [('2015-12-22',)]
```

0.3.6 Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
[21]: %sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" =_
      ↳'Success (drone ship)' AND "PAYLOAD_MASS__KG_" BETWEEN 4000 AND 6000;
```

```
* sqlite:///my_data1.db
Done.
```

```
[21]: [('F9 FT B1022',), ('F9 FT B1026',), ('F9 FT B1021.2',), ('F9 FT B1031.2',)]
```

0.3.7 Task 7

List the total number of successful and failure mission outcomes

```
[22]: %sql SELECT "Mission_Outcome", COUNT(*) FROM SPACEXTABLE GROUP BY_
      ↳"Mission_Outcome";
```

```
* sqlite:///my_data1.db
Done.
```

```
[22]: [('Failure (in flight)', 1),
      ('Success', 98),
      ('Success ', 1),
      ('Success (payload status unclear)', 1)]
```

0.3.8 Task 8

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
[23]: %sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "PAYLOAD_MASS__KG_" =_
      ↳(SELECT MAX("PAYLOAD_MASS__KG_") FROM SPACEXTABLE);
```

```
* sqlite:///my_data1.db
Done.
```

```
[23]: [('F9 B5 B1048.4',),
      ('F9 B5 B1049.4',),
```

```

('F9 B5 B1051.3',),
('F9 B5 B1056.4',),
('F9 B5 B1048.5',),
('F9 B5 B1051.4',),
('F9 B5 B1049.5',),
('F9 B5 B1060.2 ',),
('F9 B5 B1058.3 ',),
('F9 B5 B1051.6',),
('F9 B5 B1060.3',),
('F9 B5 B1049.7 ',)]

```

0.3.9 Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015. Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```

[24]: %sql SELECT substr("Date", 6, 2) AS "Month", "Landing_Outcome",
↳ "Booster_Version", "Launch_Site" FROM SPACEXTABLE WHERE "Landing_Outcome" =
↳ 'Failure (drone ship)' AND substr("Date", 1, 4) = '2015';

```

```

* sqlite:///my_data1.db
Done.

```

```

[24]: [('01', 'Failure (drone ship)', 'F9 v1.1 B1012', 'CCAFS LC-40'),
      ('04', 'Failure (drone ship)', 'F9 v1.1 B1015', 'CCAFS LC-40')]

```

0.3.10 Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```

[25]: %sql SELECT "Landing_Outcome", COUNT(*) AS "Outcome_Count" FROM SPACEXTABLE
↳ WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY
↳ "Landing_Outcome" ORDER BY "Outcome_Count" DESC;

```

```

* sqlite:///my_data1.db
Done.

```

```

[25]: [('No attempt', 10),
      ('Success (drone ship)', 5),
      ('Failure (drone ship)', 5),
      ('Success (ground pad)', 3),
      ('Controlled (ocean)', 3),
      ('Uncontrolled (ocean)', 2),
      ('Failure (parachute)', 2),
      ('Precluded (drone ship)', 1)]

```

0.3.11 Reference Links

- Hands-on Lab : String Patterns, Sorting and Grouping
- Hands-on Lab: Built-in functions
- Hands-on Lab : Sub-queries and Nested SELECT Statements
- Hands-on Tutorial: Accessing Databases with SQL magic
- Hands-on Lab: Analyzing a real World Data Set

0.4 Author(s)

Lakshmi Holla

0.5 Other Contributors

Rav Ahuja

##

© IBM Corporation 2021. All rights reserved.